

GATE CSE 2027 SYLLABUS - BOOKS AND LECTURE VIDEOS

([Full Course](#)) ([Most Repeated PYQs](#)) ([Most Asked Questions](#))

SUBJECTS	TOPICS	REFERENCES	LECTURE LINK
Engineering Mathematics	Propositional and first order logic Sets, relations, functions, partial orders and lattices Monoids, Groups. Graphs: connectivity, matching, colouring. counting, recurrence relations, generating functions. Matrices, determinants, system of linear equations, eigenvalues and eigenvectors, LU decomposition. Limits, continuity and differentiability, Maxima and minima, Mean value theorem, Integration.	[1] Ch 1: 1.1–1.4 (Propositional Logic, Applications, Propositional Equivalences, Predicates & Quantifiers) [1] Ch 2: 2.1–2.3 (Sets, Set Operations, Functions); Ch 9: 9.1, 9.5, 9.6 (Relations, Equivalence Relations, Partial Orderings) [2] Ch 2–3 (Groups, Finite Groups & Subgroups); Ch 5 (Permutation Groups, if time permits) — far more rigorous treatment of groups than any discrete-math text gives [3] Ch 1 (Fundamental Concepts — connectivity, trees); Ch 3 (Matchings & Factors); Ch 5 (Coloring) — the standard reference for graph theory rigor [4] Ch 2–3 (Permutations & Combinations, Pigeonhole); Ch 7–8 (Recurrence Relations, Generating Functions) — the gold-standard combinatorics text, deeper than any Rosen-level treatment [5] Ch 1–2 (Systems of Equations, Vector Spaces, LU Decomposition); Ch 5 (Eigenvalues & Eigenvectors); Ch on Determinants — best balance of computational depth and conceptual intuition [6] Thomas: Ch 2 (Limits & Continuity), Ch 4 (Applications of Derivatives — MVT, Maxima/Minima), Ch 5 (Integration). Spivak: Ch 5–8	Discrete Mathematics Crash Course Crash Course except Discrete

	Random variables, Uniform, normal, exponential, Poisson and binomial distributions. Mean, median, mode and standard deviation. Conditional probability and Bayes theorem.	(Limits, Continuity, Derivatives, MVT) if you want true rigor/proofs [7] Ch 2–3 (Axioms of Probability, Conditional Probability & Bayes' Theorem); Ch 4–5 (Discrete & Continuous Random Variables — Binomial, Poisson, Uniform, Normal, Exponential); Ch 6–7 (Expectation, Mean/Variance) — widely considered the best-written probability text for this exact syllabus	
Digital Logic	Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational Design of sequential circuits. Number representation and arithmetic (fixed and floating point).	[1] Ch 2 (Combinational Logic — Boolean Algebra, K-Maps); Ch 2.6 (Quine-McCluskey Tabular Method) — more rigorous derivations than Mano, good for actually understanding <i>why</i> minimization works, not just applying it [2] Ch 4 (Combinational Logic — adders, decoders, multiplexers, encoders) [2] Ch 5–6 (Synchronous Sequential Logic — flip-flops, state tables/diagrams, counters, registers) [3] Ch 1–3 (Number Representation, Fixed-Point Arithmetic); Ch 17–18 (Floating-Point Representation & Arithmetic) — this is the definitive deep-mastery text on arithmetic, well beyond what any GATE-prep book offers	Crash Course

Computer Organization and Architecture	Instruction set and addressing modes. Design of arithmetic and logic unit (ALU). Design of control unit – hardwired and microprogrammed. Memory interfacing and hierarchy: performance, cache memory mapping. I/O interface (interrupt and DMA). Instruction pipelining, pipeline hazards.	[1] Ch 2 (Instructions: Language of the Computer — full chapter, esp. addressing modes section) [1] Ch 3 (Arithmetic for Computers — ALU design, adders, multiplication/division hardware) [2] Ch 5 (Basic Computer Organization & Design — hardwired control); Ch 7 (Microprogrammed Control) — Mano remains unmatched for a clear, rigorous treatment of both control unit paradigms side by side [3] Ch 2 (Memory Hierarchy Design — cache mapping techniques, performance formulas, AMAT) — this is <i>the</i> definitive text on memory hierarchy, well beyond intro-level coverage [4] Ch 4 (Input/Output Organization — interrupts, DMA, bus arbitration) [3] Appendix C (Pipelining: Basic and Intermediate Concepts) — structural/data/control hazards, forwarding, branch prediction, covered with far more depth and precision than any GATE-level book	Crash Course
Programming and Data Structures	Programming in C. Recursion.	[1] Ch 1–6 (full core language — types, operators, control flow, functions, pointers & arrays, structures) — still the definitive text for truly understanding C, not just syntax [2] Ch 4 (Divide-and-Conquer — recurrence relations, recursion tree/master method) — treats recursion analytically rather than just procedurally,	C Crash Course Crash Course

	Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.	which is what "mastery" actually requires here [3] Ch 3 (Lists, Stacks, Queues); Ch 4 (Trees, BSTs); Ch 6 (Priority Queues/Binary Heaps); Ch 9 (Graph Algorithms) — Weiss is more rigorous than typical intro DS books, with real complexity analysis for every structure	
Algorithms	Searching, sorting, hashing. Asymptotic worst case time and space complexity. Algorithm design techniques: greedy, dynamic programming and divide-and-conquer. Graph traversals, minimum spanning trees, shortest paths.	[1] Ch 2, 6–8 (Insertion/Merge/Heap/Quick Sort, Sorting Lower Bounds, Counting/Radix/Bucket Sort); Ch 11 (Hash Tables) — CLRS gives the actual complexity proofs, not just the algorithms [1] Ch 3 (Growth of Functions — Θ, O, Ω rigorously defined); Ch 4 (Divide-and-Conquer & recurrence solving) — this is the standard for true mastery of asymptotic analysis, well beyond hand-wavy "Big-O" treatments [2] Ch 4 (Greedy Algorithms); Ch 5 (Divide and Conquer); Ch 6 (Dynamic Programming) — widely considered the best book for building genuine <i>design intuition</i> rather than memorizing standard problems, which CLRS doesn't emphasize as strongly [1] Ch 20 (Elementary Graph Algorithms — BFS/DFS); Ch 21 (MST — Kruskal's, Prim's); Ch 22 (Shortest Paths — Dijkstra, Bellman-Ford, Floyd-Warshall) — the most rigorous and complete treatment available, with full correctness proofs	Crash Course
Theory of Computation	Regular expressions and finite automata.	[1] Ch 1 (Regular Languages — DFA, NFA, regex, equivalence of NFA/DFA) — Sipser's proofs and intuitive	Crash Course

	Context-free grammars and push-down automata. Regular and context-free languages, pumping lemma. Turing machines and undecidability.	explanations are widely regarded as the clearest available for this material [1] Ch 2 (Context-Free Languages — CFGs, Chomsky Normal Form, PDAs, equivalence of CFGs/PDAs) [2] Ch 4 (Properties of Regular Languages — pumping lemma, closure properties); Ch 7 (Properties of Context-Free Languages — CFL pumping lemma) — HMU's treatment of closure properties and pumping lemma edge cases is more exhaustive than Sipser's, useful once you've built intuition from Sipser [1] Ch 3 (The Church-Turing Thesis); Ch 4 (Decidability); Ch 5 (Reducibility — undecidability, Rice's theorem) — Sipser is unmatched here for building real conceptual understanding of undecidability rather than memorized proof sketches	
Compiler Design	Lexical analysis, parsing, syntax-directed translation. Runtime environments. Intermediate code generation. Local optimisation, Data flow analyses: constant propagation, liveness analysis, common sub expression elimination.	[1] Ch 3 (Lexical Analysis); Ch 4 (Syntax Analysis — top-down/bottom-up parsing, LL, LR, LALR); Ch 5 (Syntax-Directed Translation); Ch 7 (Run-Time Environments — activation records, stack allocation, scope) [1] Ch 6 (Intermediate-Code Generation — three-address code, syntax trees, static single assignment) — the Dragon Book remains the canonical, most rigorous treatment of this topic [2] Ch 8 (Introduction to Optimization); Ch 9 (Data-Flow Analysis — liveness, reaching definitions); Ch 10 (Scalar Optimizations)	Crash Course

		— constant propagation, common subexpression elimination) — Cooper & Torczon is more implementation-focused and considerably clearer on data-flow analysis than the Dragon Book's denser treatment	
Operating System	System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.	[1] Ch 3 (Processes); Ch 4 (Threads & Concurrency); Ch 3.4/3.5 (IPC); Ch 6 (Synchronization — critical section, semaphores, monitors, classic problems) [1] Ch 8 (Deadlocks — characterization, prevention, avoidance/Banker's algorithm, detection, recovery) [2] Ch 7–10 (Scheduling — MLFQ, proportional-share, and the underlying "why" of each policy) — free, and known for building genuinely deeper intuition on scheduling trade-offs than most textbooks [2] Ch 13–15 (Address Spaces, Memory API, Address Translation); Ch 18–24 (Paging, TLBs, Swapping, Page Replacement Policies) — widely regarded as the clearest treatment of virtual memory mechanics available anywhere [2] Ch 39–44 (Files & Directories, File System Implementation, FFS, Journaling)	Crash Course
Databases	ER-model. Relational model: relational algebra, tuple calculus, SQL.	[1] Ch 7 (Entity-Relationship Model — ER diagrams, mapping constraints, weak entities, ER-to-relational mapping) [1] Ch 2 (Relational Model); Ch 6 (Formal Relational Query	Crash Course

	Integrity constraints, normal forms. File organization, indexing (e.g., B and B+ trees). Transactions and concurrency control.	Languages — relational algebra, tuple/domain relational calculus); Ch 3–4 (SQL) [2] Ch 3 (Integrity Constraints); Ch 19 (Schema Refinement — functional dependencies, 1NF–BCNF, 4NF, decomposition properties) — Ramakrishnan & Gehrke's treatment of FD-based normalization and decomposition is more mathematically precise and worked-example-driven than Silberschatz's, which matters a lot for GATE's normalization problems [1] Ch 13 (Data Storage Structures); Ch 14 (Indexing — B-tree, B+-tree structure, insertion/deletion) [1] Ch 17 (Transactions — ACID, schedules, serializability); Ch 18 (Concurrency Control — locking protocols, timestamp ordering, deadlock handling in DBs)	
Computer Networks	Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet;	[1] Ch 1 (Computer Networks and the Internet — protocol layering, circuit vs. packet switching, delay/loss/throughput analysis) [2] Ch 3 (The Data Link Layer — Hamming codes, CRC, framing); Ch 4 (The Medium Access Control Sublayer — CSMA/CD, Ethernet) — Tanenbaum's treatment of the actual error-detection math (CRC polynomial division, Hamming distance) and MAC protocol mechanics is more rigorous and worked-example-driven than Kurose & Ross	Crash Course

	Distance vector and link state routing;	[1] Ch 5 (The Network Layer: Control Plane — routing algorithms, Bellman-Ford/distance-vector, Dijkstra/link-state, hierarchical routing)	
	IPv4 - Fragmentation, CIDR Notation, Network Address Translation;	[1] Ch 4 (The Network Layer: Data Plane — IPv4 datagram format, fragmentation, CIDR addressing, NAT)	
	TCP- flow control and congestion control, socket API;	[1] Ch 3 (The Transport Layer — TCP flow control, congestion control — AIMD, slow start; socket programming)	
	DNS and HTTP.	[1] Ch 2 (The Application Layer — HTTP request/response, DNS hierarchy and resolution)	

Engineering Mathematics References:

- [1] Rosen, K. H. — *Discrete Mathematics and Its Applications* (7th/8th ed.)
- [2] Gallian, J. A. — *Contemporary Abstract Algebra* (9th/10th ed.)
- [3] West, D. B. — *Introduction to Graph Theory* (2nd ed.)
- [4] Brualdi, R. A. — *Introductory Combinatorics* (5th ed.)
- [5] Strang, G. — *Linear Algebra and Its Applications* (4th/5th ed.)
- [6] Thomas, G. B. — *Thomas' Calculus* (14th ed.) / Spivak, M. — *Calculus* (4th ed.)
- [7] Ross, S. — *A First Course in Probability* (10th ed.)

A few honest notes on trade-offs, since you specifically don't want "basics only":

- **Gallian and West are genuinely overkill in places** — they're undergrad math-major texts, not engineering-exam texts. You'll get far more rigor than GATE tests, which is good for mastery but means you'll need to be disciplined about scope (stick to the sections listed) or you'll sink time into topics (like Sylow theorems or planarity proofs) that never show up.
- **Spivak is intense** — real proof-based analysis. If you want calculus mastery *and* have the time, it's the best there is. If you're calibrating for GATE efficiency, Thomas' Calculus with careful attention to the MVT/continuity proofs is the more balanced pick.
- **Strang over Lay/Kreyszig**: Strang is chosen because his treatment of LU decomposition and eigenvalues builds genuine geometric intuition, not just mechanical computation — which is what "mastery" usually means for linear algebra.

Digital Logic References:

- [1] Katz, R. H. & Borriello, G. — *Contemporary Logic Design* (2nd ed.)

- [2] Mano, M. M. & Ciletti, M. D. — *Digital Design: With an Introduction to the Verilog HDL* (5th/6th ed.)
[3] Parhami, B. — *Computer Arithmetic: Algorithms and Hardware Designs* (2nd ed.)

Notes on trade-offs:

- **Katz over Mano for Boolean algebra/K-maps:** Mano is the more common GATE recommendation and is perfectly sufficient, but Katz builds more intuition for *why* the tabular method generalizes K-maps — worth it if you want real mastery rather than pattern-recognition.
- **Mano stays the pick for circuit design** — it's genuinely excellent for combinational/sequential design and there's no need to go elsewhere; the "basics-only" concern doesn't really apply here since Mano's treatment is already rigorous and complete for this scope.
- **Parhami is a significant step up** from what GATE strictly requires — it's a graduate-level computer arithmetic text. For fixed-point, stick close to the listed chapters; the floating-point chapters will give you a much deeper grip on IEEE 754 mechanics (rounding, guard bits, normalization) than any standard COA book.

COA References

- [1] Patterson, D. A. & Hennessy, J. L. — *Computer Organization and Design: The Hardware/Software Interface* (5th ed., RISC-V or MIPS edition)
[2] Mano, M. M. — *Computer System Architecture* (3rd ed.)
[3] Hennessy, J. L. & Patterson, D. A. — *Computer Architecture: A Quantitative Approach* (6th ed.)
[4] Hamacher, C., Vranesic, Z., & Zaky, S. — *Computer Organization and Embedded Systems* (6th ed.)

Notes on trade-offs:

- **Hennessy & Patterson's Quantitative Approach is genuinely graduate-level** — it's the book real computer architects use, not a GATE book. For memory hierarchy and pipelining specifically this pays off, because GATE questions on cache mapping and pipeline hazards often test the kind of precise reasoning this book builds (as opposed to memorized formulas). Just stay disciplined about scope — don't wander into VLIW, superscalar, or multiprocessor chapters unless you want extra depth for its own sake.
- **Mano stays the best pick for control unit design** specifically because most modern COA books (including Patterson & Hennessy) barely touch microprogrammed control anymore — it's considered "classical," but GATE still tests it heavily, so an older, more thorough text is actually the *better* choice here, not a compromise.
- **Hamacher for I/O** is chosen because its treatment of interrupt-driven I/O and DMA controller mechanics is more concrete and hardware-grounded than Patterson & Hennessy's higher-level treatment.

Programming and Data Structures References

- [1] Kernighan, B. W. & Ritchie, D. M. — *The C Programming Language* (2nd ed., ANSI C)
[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. — *Introduction to Algorithms* (CLRS, 3rd/4th ed.)
[3] Weiss, M. A. — *Data Structures and Algorithms in C* (2nd ed.)

Notes on trade-offs:

- **K&R over any modern "Programming in C" textbook:** it's terse and assumes some mathematical maturity, but it's written by the language's creators and teaches you to actually *think* in C (pointer arithmetic, memory layout) rather than pattern-matching syntax — which is exactly the depth GATE's tricky C-output questions require.
- **CLRS for recursion specifically:** you don't need the whole book here, just the recurrence-solving toolkit (substitution method, recursion trees, master theorem) — this is the single most GATE-relevant skill for recursion, more than tracing code by hand.
- **Weiss over CLRS for the data structures themselves:** CLRS is more mathematically dense and algorithm-proof-heavy; for graph/tree/heap *implementation and behavior* (what GATE actually tests), Weiss strikes a better balance of rigor and directness. You'll want CLRS again later for Algorithms (sorting, greedy, DP, graph algorithms) — I can fold that in separately if that's a syllabus section for you.

Algorithms References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. — *Introduction to Algorithms* (CLRS, 3rd/4th ed.)
[2] Kleinberg, J. & Tardos, É. — *Algorithm Design* (1st ed.)

Notes on trade-offs:

- **CLRS carries most of this subject**, and deliberately so — it's the most rigorous algorithms text in common use, with actual correctness proofs for every algorithm (loop invariants, exchange arguments, etc.), which is what separates "I can trace Dijkstra by hand" from "I understand why greedy works here." The cost is density — CLRS is proof-heavy and can be slow going.
- **Kleinberg & Tardos for design techniques specifically:** this is the one place I'd deviate from CLRS. K&T's chapters on greedy/DP/divide-and-conquer are built around teaching you *how to recognize which technique fits a new problem* — a skill CLRS doesn't focus on since it's organized by algorithm, not by design paradigm. This matters a lot for GATE's "here's a new problem, design an algorithm" style questions.
- If CLRS's density becomes a bottleneck given your GATE 2027 timeline, *Algorithms* by Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani is a leaner, still-rigorous alternative that covers all four rows above in one shorter book — worth considering if you want one book instead of two.

Theory of Computation References

- [1] Sipser, M. — *Introduction to the Theory of Computation* (3rd ed.)
[2] Hopcroft, J. E., Motwani, R., & Ullman, J. D. — *Introduction to Automata Theory, Languages, and Computation* (3rd ed.)

Notes on trade-offs:

- **Sipser is the primary spine** for this entire subject, and deliberately so — it's consistently ranked as the best-written TOC textbook precisely because it builds genuine intuition (through his famous "proof idea" sections) rather than just formal proofs. For a subject as abstract as this, that intuition is what "mastery" actually means.

- **HMU (Hopcroft-Motwani-Ullman) only for the pumping lemma/closure properties row** — it's more exhaustive and formal than Sipser, which helps once you already have the Sipser-level intuition and want to handle tricky "is this language regular/context-free" GATE questions with full rigor. Using HMU as your primary text from the start would be a mistake — it's drier and less intuitive for first-pass learning.
- If you find Sipser's proof style still too terse in places, *Elements of the Theory of Computation* by Lewis & Papadimitriou is a solid, more procedural alternative — but for genuine mastery (not just exam-passing), Sipser remains the stronger choice.

Compiler Design References

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. — *Compilers: Principles, Techniques, and Tools* (2nd ed., "The Dragon Book")
- [2] Cooper, K. D. & Torczon, L. — *Engineering a Compiler* (2nd ed.)

Notes on trade-offs:

- **Dragon Book for front-end (lexing, parsing, syntax-directed translation, intermediate code)**: it's the canonical, most cited text in the field, and its treatment of parser construction (especially LR/LALR table construction) is more rigorous and complete than any alternative — worth pushing through the density for.
- **Cooper & Torczon for optimization specifically** — this is a deliberate departure from the Dragon Book, whose optimization chapters are notoriously terse and hard to self-study from. Cooper & Torczon was written specifically to teach optimization and data-flow analysis clearly, with worked examples of the exact techniques your syllabus lists (constant propagation, liveness analysis, CSE) — genuinely better for mastery than gritting through the Dragon Book here.
- If the Dragon Book's front-end chapters feel too dense on a first pass, *Engineering a Compiler* also covers lexing/parsing (Ch 2–3) reasonably well and could serve as your sole book for the whole subject — a leaner one-book option if your GATE 2027 timeline is tight.

OS References

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. — *Operating System Concepts* (10th ed.)
- [2] Arpaci-Dusseau, R. H. & Arpaci-Dusseau, A. C. — *Operating Systems: Three Easy Pieces* (freely available at ostep.org)

Notes on trade-offs:

- **Silberschatz for processes, threads, IPC, synchronization, and deadlock** — its coverage here is comprehensive and precise, and these are the areas where GATE tests fairly standard, well-defined concepts (semaphore code-tracing, Banker's algorithm), which Silberschatz explains rigorously.
- **OSTEP (Three Easy Pieces) for scheduling, memory management, and file systems** — this is a deliberate departure from the "standard" GATE recommendation. OSTEP is written by researchers specifically to build *intuition* for why these mechanisms exist (e.g., why page replacement policies trade off differently, why journaling solves crash consistency), which is exactly the depth you're asking for beyond rote formulas. It's also free and legally available at ostep.org, so worth grabbing directly.

- If you'd rather stick to one book across the whole subject for consistency, Silberschatz alone is sufficient and standard — but for actual mastery of scheduling and memory management specifically, OSTEP is the stronger pick.

DBMS References

- [1] Silberschatz, A., Korth, H. F., & Sudarshan, S. — *Database System Concepts* (7th ed.)
 [2] Ramakrishnan, R. & Gehrke, J. — *Database Management Systems* (3rd ed.)

Notes on trade-offs:

- **Silberschatz carries most of this subject** and is the standard, highly rigorous reference — its coverage of relational algebra/calculus, indexing, and transactions/concurrency control is thorough and precise, exactly matching the GATE syllabus's scope.
- **Ramakrishnan & Gehrke only for normalization** — this is the one deliberate swap. Their chapter on functional dependencies and normal forms is built around more worked decomposition examples (BCNF decomposition algorithms, lossless-join/dependency-preserving checks) than Silberschatz provides, and GATE's normalization questions are usually computational rather than conceptual — so the extra worked practice matters more than which book is "more standard."
- Both books overlap heavily elsewhere, so you don't strictly need both — but if you only buy one, Silberschatz is the safer single choice; add Ramakrishnan & Gehrke specifically if normalization is a weak spot.

CN References

- [1] Kurose, J. F. & Ross, K. W. — *Computer Networking: A Top-Down Approach* (7th/8th ed.)
 [2] Tanenbaum, A. S. & Wetherall, D. J. — *Computer Networks* (5th ed.)

Notes on trade-offs:

- **Kurose & Ross carries most of this subject** — its top-down structure (starting from the application layer down) builds strong conceptual intuition, and its treatment of TCP congestion control, IPv4 addressing/NAT, and routing algorithms is exactly matched to what GATE tests, with clear worked examples.
- **Tanenbaum specifically for the data link layer** — this is the deliberate swap. Kurose & Ross treats error detection and MAC fairly briefly since it's application-first in philosophy; Tanenbaum, being more bottom-up and hardware-grounded, walks through CRC computation and CSMA/CD timing analysis in much more depth — and GATE's data-link questions (computing CRC remainders, Hamming code correction, Ethernet timing) reward exactly that kind of mechanical fluency.
- If you'd rather use one single book for the whole subject for consistency, Kurose & Ross alone is the standard, sufficient choice — add Tanenbaum's data-link chapters specifically if CRC/Hamming-code problems are a weak spot.